

TAP v1 — Trajectory Advantage Predictor: Implementation Report

Wave 2: integration + Prime Intellect hardening + handoff

PDF engine: reportlab 5.0.0 (build_report_pdf.py); no LaTeX toolchain on the ARM64 build host, so report.tex ships beside this reportlab-rendered PDF.

1. Goal

TAP predicts one scalar, `predicted_utility_points`, for a candidate GRPO update batch on a Qwen3-8B LoRA policy: how much applying it improves held-out MATH while lightly penalizing unrelated drift. Positive helps, ~ 0 has little effect, negative hurts. Hypothesis: a candidate's usefulness is predictable from its reward, familiarity, gradient direction, current policy state, and similarity to recently reinforced updates. Success = beat random and probability-only on held-out states; the strong result also beats reward-only and the no-history model.

2. Architecture

Three layers, separated so everything except GPU collection runs on CPU. (1) Data collection (`math_loop/`) on a 4-5xH100 Prime Intellect pod: `tap_controller` walks 2 chains x 6 states x 6 candidates, branching each candidate from the byte-identical before-state and advancing the main chain with a seeded-random candidate; `branch.py` runs one GRPO step via `prime-rl resume` and writes raw artifacts; `features.py` emits the four Parquet tables; `tap_probes.py` computes probe NLL and generic KL drift. (2) Launcher + safety (`run_prime_rl_math_loop.py`, `reap_pods.py`), Wave 2 (section 7). (3) TAP model + evaluation (`tap/`) on CPU: `dataset/splits`, `SmallTAP`, `baselines`, `training`, `within-state ranking eval`, and the `run_all` entry point that writes `results.csv` + `report.{md, tex, pdf}`.

3. Data schema & label

Four Parquet tables enforced by `tap/schema.py --validate`: `states` (`ids`, `step`, `seed`, `hashes`, `lr`, `grpo_beta`, `lora_rank`, `before-probes`, Adam moment norms, 16-value policy fingerprint, `history ids`); `trajectories` (`rewards`, `advantage`, `log-prob/entropy stats and quantiles`, `confidence slope`, `log-ratios`, `clipped fraction`, `embedding`); `candidates` (the main training table: `reward/advantage moments`, `probability stats`, 256-d `embedding`, 64-d `gradient sketch`, norms, `history similarities`, `after-probes`, `gains`, `utility_points`, `exact-match diagnostics`, `is_selected_for_main_chain`); `history` (`last-4 updates`).

$$\text{utility_points} = 1000 * (0.8 * \text{matched_gain} + 0.2 * \text{global_gain} - 0.03 * \max(\text{incremental_generic_kl}, 0));$$
 gains are NLL-before minus NLL-after, in nats per non-padding token. Exact-match is diagnostic only.

4. Features

State: `step`, `lr`, `grpo_beta`, Adam moment norms, policy fingerprint. Candidate: `reward/advantage mean+std`, `mean/geometric/arithmetic probability`, `entropy stats`, `log-prob quantiles`, `early-late confidence slope`, `sequence length`, `policy/reference log-ratio`, 256-d `embedding`, 64-d `gradient sketch`, `gradient norm`, `estimated update norm`. History: `last-4 embeddings + sketches`, `relative ages`, `reward/probability stats`, `semantic and gradient similarity`. Probability = familiarity; `reward/advantage` = desirability; `gradient` = expected parameter movement; `history similarity` = already reinforced.

5. Models

`SmallTAP`: candidate-embedding projection 256->64, gradient-sketch 64->32, numeric MLP 16->64, state MLP 26->32, history projected to 64, one 4-head candidate->history cross-attention layer, two-layer MLP head

(hidden 128), one scalar output. Trainable parameters: 109,537 (under the 250k target). Baselines: random, reward/advantage mean, geometric/arithmetic probability, reward x surprisal, semantic novelty, gradient norm/alignment (heuristic); ridge, gradient-boosted trees, no-history MLP, numeric-only, candidate-only (learned). If the attention model does not beat the simpler learned baselines on the real collection, the simpler model is reported as TAP v1.

Training: Huber on standardized utility + 0.5 x within-state pairwise ranking loss + small weight decay. Split: train chain 0 / test chain 1, swap, average; all candidates from one state stay in one split.

6. Evaluation (synthetic placeholder)

The GPU collection is blocked on the Prime Intellect API key, so labels do not yet exist. The table is the plumbing-check output of `tap.run_all` on the synthetic 72-label dataset (`outputs/tap_synth_72`) — not a scientific result.

Model	Spearman	Pair acc	Mean util	vs rand	vs reward	vs prob
TAP (SmallTAP)	0.738	0.817	34.42	+39.35	+12.57	+48.30
ridge	0.829	0.861	35.79	+40.72	+13.94	+49.67
candidate-only	0.843	0.872	36.59	+41.52	+14.74	+50.46
no-history MLP	0.481	0.683	32.92	+37.85	+11.07	+46.80
reward-only	0.233	0.594	21.85	+26.78	0.00	+35.73
prob-only (geo)	-0.148	0.444	-13.88	-8.95	-35.73	0.00
random	0.086	0.533	-4.93	0.00	-26.78	+8.95

On synthetic data TAP beats random, reward-only, and probability-only (`beat_random/beat_reward/beat_prob` all True); ridge and candidate-only edge it — the comparison the real 72-label run must resolve.

7. Wave 2 status

CPU gate PASS: `py_compile` of `math_loop/*.py` + `tap/*.py`; 55 TAP unit tests; `tap_controller --dry-run`; `tap.run_all` on the synthetic set. Launcher hardened + 15 tests (including a SIGTERM-teardown subprocess test): `lambdalabs` default; required `--prime-rl-commit`; a fail-closed cost monitor and an independent wall-clock deadline that reap the pod and exit non-zero on breach or monitor failure; `pod_id.txt` on create; `atexit` + SIGINT/SIGTERM + breach teardown reaping by `tap-v1-smoke-` prefix and by id; `--keep-pod` forbidden; `--gpu-count>1` and any non-smoke run gated behind `TAP_ALLOW_FULL_RUN=1`; SSH via the provided `/workspace/private_key.pem`. `reap_pods.py` refuses empty or non-`tap-v1-` prefixes.

Smoke BLOCKED: prime availability list returns 'No API key configured'; `PRIME_API_KEY` is absent. The user must supply it (`RUNBOOK.md`). Integration finding: `tap_controller.run_controller` stubs real runs; the GPU driver loop must be enabled in that read-only file before labels are produced; the launcher fails fast and reaps the pod if hit.